[Trung Nguyen] C Programing Review

Part 1: Linux System Programming

1) Variabless

- A variable is a name given to a piece of data stored in memory at some address.
- A variable name may use any character in the ranges `0-9`, `A-Z`, `a-z`, plus the underscore, subject to the following rules: it must not begin with a digit, must not begin with two underscores, and must not begin with an underscore followed by an uppercase letter.
- Using a variable's type: before a variable can be used it must be declared, telling C what its type is. Once declared, the type cannot change at run time. The assigned type remains in effect until the variable goes out of scope and the storage is reused.
- Uninitialized variables hold indeterminate values.
- **Boolean type:** in C, `0` means *false* and any non-zero value means *true*.

Static / Extern

`static` is a **storage-class specifier** applied to a variable or function at its declaration. It tells the compiler one of two things, depending on where it is placed: **restrict this name to the current translation unit**, or **give this variable a lifetime spanning the whole program instead of destroying it when the function returns**.

When to use `static` in C

A variable or function should be declared `static` in three cases:

- **A function used only inside one file** — a helper that is not part of the module's public API and is not declared in the `.h` file.
- **A global variable owned by a single module** — internal state that other files must not touch.
- **A local variable that must retain its value between calls** — counters, "already initialized" flags, reusable buffers.

`extern` is likewise a **storage-class specifier** applied at declaration. It tells the compiler that **this variable or function is defined in another translation unit** — accept the use, and let the linker resolve it.

When to use `extern` in C

`extern` is used when you need to access something defined elsewhere. In practice there are only two cases:

- **Declaring a shared global variable in a `.h` file** — the variable is defined exactly once in a `.c` file, and other files include the header to use it.
- **Declaring a function** — writing `extern` is not actually required, since functions have external linkage by default.

Operator Precedence and Associativity

Associativity determines the order in which operators of the same precedence level are evaluated.

Precedence	Operator	Description	Associativity
1	`++` `--`	Suffix/postfix increment and decrement	Left-to-right
1	`()`	Function call	Left-to-right
1	`[]`	Array subscripting	Left-to-right
1	`.`	Structure and union member access	Left-to-right
1	`->`	Structure and union member access through pointer	Left-to-right
1	`(type){list}`	Compound literal (C99)	Left-to-right
2	`++` `--`	Prefix increment and decrement	Right-to-left
2	`+` `-`	Unary plus and minus	Right-to-left
2	`!` `~`	Logical NOT and bitwise NOT	Right-to-left
2	`(type)`	Cast	Right-to-left

Precedence	Operator	Description	Associativity
2	`*`	Indirection (dereference)	Right-to-left
2	`&`	Address-of	Right-to-left
2	`sizeof`	Size-of	Right-to-left
2	`_Alignof`	Alignment requirement (C11)	Right-to-left
3	`*` `/` `%`	Multiplication, division, and remainder	Left-to-right
4	`+` `-`	Addition and subtraction	Left-to-right
5	`<<` `>>`	Bitwise left shift and right shift	Left-to-right
6	`<` `<=` `>` `>=`	Relational operators	Left-to-right
7	`==` `!=`	Equality operators	Left-to-right
8	`&`	Bitwise AND	Left-to-right
9	`^`	Bitwise XOR (exclusive or)	Left-to-right
10	`|`	Bitwise OR (inclusive or)	Left-to-right
11	`&&`	Logical AND	Left-to-right
12	`||`	Logical OR	Left-to-right
13	`?:`	Ternary conditional	Right-to-left
14	`=` `+=` `-=` `*=` `/=` `%=` `<<=` `>>=` `&=` `^=` `|=`	Assignment operators	Right-to-left
15	`,`	Comma	Left-to-right

Mnemonic for operator precedence and associativity: PUMA'S REBL TAC

https://en.cppreference.com/c/language/operator_precedence

where P = Postfix, U = Unary, M = Multiplicative, A = Additive, S = Shift, R = Relational, E = Equality, B = Bitwise, L = Logical, T = Ternary, A = Assignment, and C = Comma.

Example 1

```
int main()
{
    int x, y = 0, z = 0;
    x = x * 3 / 5;
    y = x / 5 * 3;
    z = (float)(3 / 5) * x;
```

```
    printf("x=%d , y = %d", x, y, z);
```

Pitfalls to avoid in the code above:

- `x` is used before being initialized, so it holds a garbage value and the result is meaningless — in some cases an outright bug.
- For large values of `x`, the intermediate product `x * 3` can exceed the range of a 32-bit `int` and overflow.
- `x` and `y` will not agree in general, because integer division truncates the remainder, and the two expressions divide at different points.
- `(float)(3 / 5)` casts *after* the integer division has already produced `0`. If the intent was real-valued arithmetic, the operands must be floating point from the start:

```
float r = (3.0f / 5.0f) * (float)x;
```

Example 2

```
char buf[8];
printf("What is your name?\n");
scanf("%s", buf);
```

Pitfalls to avoid in the code above:

- `%s` has **no length limit**. Typing 20 characters writes 21 bytes (including the terminating `'\0'`) into an 8-byte buffer. The overflow spills onto the stack frame — other variables, saved registers, and possibly critical memory.
- **`%s` stops at whitespace.** Entering `My Name Is A` reads only `My`; the rest stays in the input stream and is picked up by the next call.
- The return value of `scanf` — the number of items successfully converted — is not checked.
- The corrected version:

```
char buf[8];
if (scanf("%7s", buf) != 1) {
    /* handle error */
}
```

Example 3

```
float x = 1e20f;
float z = (x * 1e30f) / 1e20f;
float m = (x / 1e20f) * 1e30f;
printf("x = %f\n", x);
```

```
printf("z = %f\n m = %f\n", z, m);


printf("k = %f", 2 / 3);
```

Example 4

```
void test1(void)
{
    uint8_t buf[] = {0x10, 0x20, 0x30, 0x40};
    uint8_t *p = buf;
    int n = 3, m = 5;

    int r = *p++ + n * m % 4 >> 1 & 0x0F ? ++n : --m;
    printf("r: %d, n : %d, m: %d, *p : 0x %0x ", r, n, m, *p);
}

int main()
{
    test1();
    return 0;
}
```

An example used to distinguish precedence levels.

Operator precedence order here: `++` (postfix), `*` (dereference), `*` `/`, `+`, `>>`, `&`, `?:`

Analysis:

- `p++` yields the old value and only then increments, so `*p++` reads `0x10`.
- `n * m % 4` = `3 * 5 % 4` = `3`, then `3 + 0x10` = ...

2) Functions and Pointers

Functions

A function is a C language construct that associates a compound statement (the function body) with an identifier (the function name). Every C program begins execution at `main`, which then terminates or calls other functions, either user-defined or from a library.

A function is introduced by a function declaration or a function definition.

Functions may take zero or more parameters, initialized from the arguments of the function-call operator, and may return a value to the caller via a `return` statement.

Function declaration. A function declaration introduces an identifier that designates a function and, optionally, specifies the types of its parameters (a prototype). Unlike a definition, a declaration may appear at block scope as well as at file scope.

If a **function declaration** appears outside of any function, the **identifier** it declares has **file scope** and **external linkage**, unless `static` is used or a prior `static` declaration is visible.

If the declaration appears **inside another function**, the identifier has **block scope** and either **internal linkage** or **external linkage**.

Example:

```
/* left-hand example */
void foo(void);

int main(void)
{
    foo();
}
```

```
/* right-hand example */
static void foo(void);

int main(void)
{
    foo();
}
```

In the left-hand example the function is visible to and usable from other translation units; in the right-hand example it is visible only within the current file.

Any parameter of array type is **adjusted** to the corresponding pointer type. That pointer type may carry qualifiers if those qualifiers appear between the square brackets `[]` of the array declaration.

```
int f(int[]);                 // declares int f(int*)
int g(const int[10]);         // declares int g(const int*)
int h(int[const volatile]);   // declares int h(int * const volatile)
int x(int[*]);                // declares int x(int*)
```

```
void func(int arr[]);
void func(int *arr);
```

The two forms above are equivalent as function parameter declarations. Consequently, when `const` is involved, the equivalence is:

```
void func(int arr[const]);
void func(int * const arr);
```

Inline Functions

The purpose of the `inline` specifier is to give the compiler a hint to perform optimizations such as *function inlining*, which normally requires the function definition to be available at the call site. The compiler may — and often does — ignore the presence or absence of `inline` when optimizing.

If the compiler does perform **function inlining**, it replaces **a call to the function** with **the function body**, avoiding **function-call overhead** (pushing data onto the stack, retrieving the return value). This can make **the executable larger**, because the function's code is duplicated at every call site.

The effect resembles a **function-like macro**, except that **identifiers** and **macros** used inside the function refer to the definitions **visible at the point of the function definition**, not at the point of the call.

```
// inline
inline int add(int a, int b)
{
    return a + b;
}

int x = add(10, 20);
/* after inlining: */
int x = 10 + 20;
```

Regardless of whether inlining actually takes place, the following semantic rules for inline functions always hold:

- Any function with **internal linkage** may be declared `static inline` with **no further restrictions**.
- An `inline` function **without** `static` must not **define a non-**`const` **function-local static variable**, and must not **refer to a static variable with file scope**.

```
static inline int add(int a, int b)
{
    return a + b;
}
```

Permitted.

Caveat 1

```
inline int counter(void)
{
    static int count = 0;
    return ++count;
}
```

Not permitted, because `count` is a **non-`const` function-local static variable**.

The reason relates to the case where the compiler actually inlines the function: if the body is pasted into several places, the question of whether all those inlined copies refer to **one single static object** becomes significant.

Caveat 2

```
static int value = 10;


inline int get_value(void)
{
    return value;
}
```

Ill-formed: `value` has **file scope and internal linkage**, while `get_value` is declared `inline` (without `static`). Therefore `get_value()` **must not refer to `value`** under the rule described above.

3) Pointers

A **pointer** is an **object type** used to hold the address of — that is, to reference — **a function** or **an object of some other type**. A pointer may also refer to no object or function at all; this case is represented by the **null pointer value**.

Pointers implement **indirection**, one of the most widely used techniques in programming. Pointers can be used to:

- implement **pass-by-reference semantics**;
- access objects with **dynamic storage duration**;
- implement **"optional" types** using the **null pointer value**;
- express **aggregation relationships** between structs;
- implement **callbacks** using **function pointers**.

Example 5

```
int main()
{
    int i = 0, j = 1;
    int x[3][4] = {
        {0, 1,  2,  3},
        {4, 5,  6,  7},
        {8, 9, 10, 11}
    };

    int y = x[++i, ++j];
    printf("j = %d \n", j);
    printf("y = %d \n", y);

    j = 1;
    printf("*(x + (++j) )   = %d \n", *(x + (++j) ) );
    printf("%d", &x[2][0]);
}
```

Error analysis

`int y = x[++i, ++j];`

`x[++j]` has type `int *`, not `int`. Assigning a pointer to an `int` variable is a type error.

`printf("%d", *(x + (++j)));`

`*(x + (++j))` is also an `int *`. Printing a pointer with `%d` is undefined behavior.

`printf("%d", &x[2][0]);`

Likewise — `&x[2][0]` is an `int *` and cannot be printed with `%d`.

Fully corrected version

```
#include <stdio.h>

int main(void)
{
    int i = 0, j = 1;
    int x[3][4] = {
        {0, 1,  2,  3},
        {4, 5,  6,  7},
        {8, 9, 10, 11}
    };
    int y = x[++i, ++j][0];           /* added [0] to strip the last level */
    printf("i = %d, j = %d\n", i, j); /* i = 1, j = 2 */
    printf("y = x[2][0] = %d\n", y);  /* y = 8        */
    printf("\n");

    j = 1;
    printf("x            = %p  (pointer to a row)\n", (void *)x);
    printf("x + 2        = %p  (skips 2 rows = 32 bytes)\n", (void *)(x + 2));
    printf("*(x + 2)     = %p  (pointer to first element of row 2)\n", (void *)*(x + 2));
    printf("*(*(x + 2))  = %d  (the actual value)\n", *(*(x + 2)));
    printf("\n");

    printf("&x[2][0]     = %p\n", (void *)&x[2][0]);
    printf("*(x + 2)     = %p  (identical to the line above)\n", (void *)*(x + 2));
    printf(" x[2][0]     = %d\n", x[2][0]);
    printf("*(*(x+2)+0)  = %d  (identical to the line above)\n", *(*(x + 2) + 0));
    printf("\n");

    printf("sizeof(x)      = %zu  (whole 3x4 array)\n", sizeof x);
    printf("sizeof(x[0])   = %zu  (one row of 4 ints)\n", sizeof x[0]);
    printf("sizeof(x[0][0])=%zu  (one element)\n", sizeof x[0][0]);
    return 0;
}
```

Program output

```
$ ./array_pointer
i = 1, j = 2
```

```
y = x[2][0] = 8

x            = 0x7fff9472b660   (pointer to a row)
x + 2        = 0x7fff9472b680   (skips 2 rows = 32 bytes)
*(x + 2)     = 0x7fff9472b680   (pointer to first element of row 2)
*(*(x + 2))  = 8   (the actual value)

&x[2][0]     = 0x7fff9472b680
*(x + 2)     = 0x7fff9472b680   (identical to the line above)
 x[2][0]     = 8
*(*(x+2)+0)  = 8   (identical to the line above)

sizeof(x)      = 48  (whole 3x4 array)
sizeof(x[0])   = 16  (one row of 4 ints)
sizeof(x[0][0])=4   (one element)
```

Effective Type

The effective type of a memory region is the type the compiler considers that region to hold. Accessing the region through a different type violates the strict aliasing rule, and the result is undefined.

Effective type is determined in two ways.

Memory created by a declaration

```
int x;              /* effective type: int     */
uint8_t buf[64];    /* effective type: uint8_t */
```

This type is fixed and never changes.

Memory obtained from an allocation function (`malloc`, `calloc`, `realloc`) or referred to through a `void *` has **no declared type**. It acquires an effective type in the following ways:

- By storing a value of a given type through a pointer to that type:

```
void *p = malloc(sizeof(int));
*(int *)p = 42;
int v = *(int *)p;
```

- By `memcpy` or `memmove` from another region:

```
int src = 42;
void *dst = malloc(sizeof(int));
memcpy(dst, &src, sizeof src);
int v = *(int *)dst;
```

Here `dst` inherits the effective type `int` from `src`.

Strict Aliasing — Access Rules

Given a memory region whose effective type is T1, accessing it through a different type T2 is undefined behavior. Strict aliasing lets the compiler assume that two pointers of different types never refer to the same memory. That assumption is what allows it to keep a value in a register instead of reloading it from memory after every store.

Five exceptions are permitted:

- T2 is a type compatible with T1
- T2 is T1 with added `const` / `volatile` qualifiers
- T2 is the signed or unsigned counterpart of T1
- T2 is a struct or union that **contains** a member of a type compatible with T1 (including nested members)
- T2 is a character type: `char`, `signed char`, `unsigned char`

Example

```
#include <stdio.h>
#include <stdint.h>

uint32_t bits_of_float(float f)
{
    return *(uint32_t *)&f;
}

int main(void)
{
    printf("%08X\n", bits_of_float(1.0f));
}
```

`f` is declared as a `float`, so its effective type is `float`. The expression `*(uint32_t *)&f` reads that memory through the type `uint32_t` — a strict aliasing violation.

Correct version

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

uint32_t bits_of_float(float f)
{
    uint32_t bits;
    memcpy(&bits, &f, sizeof bits);
    return bits;
}

int main(void)
{
    printf("%08X\n", bits_of_float(1.0f));
}
```

4) Structures, Unions and typedefs; Passing Structs by Value vs. by Pointer

Structures

Within a struct object, the addresses of its elements (and the addresses of the bit-field allocation units) increase in the order in which the members were defined. A pointer to a struct can be cast to a pointer to its first member (or, if that member is a bit-field, to its allocation unit). Likewise, a pointer to the first member of a struct can be cast to a pointer to the enclosing struct. There may be unnamed padding between any two members of a struct or after the last member, but never before the first member. The size of a struct is at least as large as the sum of the sizes of its members.

All members that are not initialized explicitly are empty-initialized.

Padding and Alignment in a Struct

The compiler inserts filler bytes between members so that each member lands at an address divisible by its own size. The CPU reads and writes correctly aligned data faster, and many architectures — Cortex-M0, for instance — cannot perform a misaligned access at all.

Two rules:

- **Each member** starts at an address divisible by its size.
- **The struct as a whole** has a size divisible by the alignment of its largest member, so that arrays of the struct remain correctly aligned.

```
#include <stdio.h>
#include <stdint.h>

struct A {
    uint8_t  a;
    uint32_t b;
    uint8_t  c;
};

struct B {
    uint32_t b;
    uint8_t  a;
    uint8_t  c;
};

int main(void)
{
    printf("sizeof(A) = %zu\n", sizeof(struct A));
    printf("sizeof(B) = %zu\n", sizeof(struct B));
}
```

Output:

```
trung@trungnm:~/Documents/session2/a$ ./padding
sizeof(A) = 12
sizeof(B) = 8
```

A bug caused by padding

```
#include <stdio.h>
#include <string.h>
#include <stdint.h>

struct Point {
```

```
    uint8_t  x;
    uint32_t y;
};

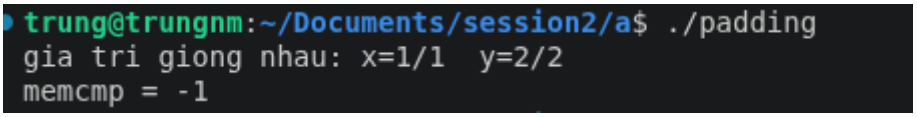
int main(void)
{
    struct Point p1, p2;

    memset(&p1, 0, sizeof p1);
    p1.x = 1;
    p1.y = 2;

    p2.x = 1;
    p2.y = 2;

    printf("same values: x=%u/%u y=%u/%u\n", p1.x, p2.x, p1.y, p2.y);
    printf("memcmp = %d\n", memcmp(&p1, &p2, sizeof p1));
}
```

Output:



Reason: `memcmp` compares every byte, including the three padding bytes between `x` and `y`. `p1` was cleared with `memset`, so its padding is zero; the padding in `p2` still holds leftover garbage from the stack. The two structs are equal member for member, yet `memcmp` reports them as different.

A way to work around padding is `__attribute__((packed))`, a compiler extension that asks the compiler to insert no padding at all.

The C standard is explicit on this point: **the value of padding bytes is indeterminate** — even after every member has been assigned.

Flexible Array Member

Definition. The last member of a struct, declared without a size. Its actual length is decided when the memory is allocated.

Mandatory rules:

- It must be the **last** member.

- The struct must have **at least one other member** before it.
- It **does not count toward** `sizeof`.

```
#include <stdio.h>
#include <stdlib.h>

struct Packet {
    int  length;
    char data[];
};

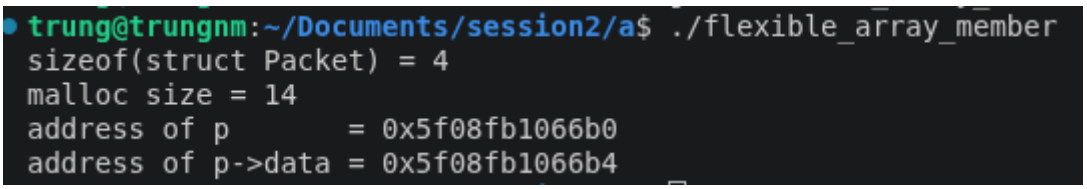
int main(void)
{
    printf("sizeof(struct Packet) = %zu\n", sizeof(struct Packet));

    struct Packet *p = malloc(sizeof(struct Packet) + 10);
    if (p == NULL)
        return 1;

    printf("malloc size = %zu\n", sizeof(struct Packet) + 10);
    printf("address of p = %p\n", (void *)p);
    printf("address of p->data = %p\n", (void *)p->data);

    free(p);
    return 0;
}
```

Output:



5) Dynamic Memory Allocation in C

Definition. Dynamic allocation means requesting memory from the heap while the program is running, instead of letting the compiler decide the size at compile time.

Use it when the size of the data is only known at run time, or when the data must outlive the scope of the function that created it.

Four functions in `<stdlib.h>`:

Function	Purpose
`malloc(n)`	Request `n` bytes; the contents are garbage
`calloc(n, size)`	Request `n * size` bytes, zero-initialized
`realloc(p, n)`	Resize an existing block to `n` bytes
`free(p)`	Return the block to the system

Problems to Avoid

Bug 1 — Memory leak

```
#include <stdio.h>
#include <stdlib.h>

void create_memory(void)
{
    int *p = malloc(100 * sizeof(int));
    if (p == NULL)
        return;

    p[0] = 10;
    printf("p[0] = %d\n", p[0]);
}

int main(void)
{
    for (int i = 0; i < 100000; i++)
    {
        create_memory();
    }

    printf("Done\n");
    return 0;
}
```

The memory is requested but never returned, and the only pointer to it is lost when the function returns — a leak. Each block stays allocated for the lifetime of the process, until memory runs out.

Bug 2 — Overwriting a pointer before freeing it

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *p = malloc(64);
    if (p == NULL)
        return 1;

    printf("Allocated 64 bytes\n");

    p = malloc(128);
    if (p == NULL)
        return 1;

    printf("Allocated 128 bytes\n");

    free(p);
    return 0;
}
```

Another cause of a leak: the pointer is reassigned while the first block is still allocated, so that block can no longer be reached or freed.

Bug 3 — Dangling pointer (use after free)

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *p = malloc(64);
    if (p == NULL)
        return 1;

    p[0] = 'A';
```

```
    printf("Before free: %c\n", p[0]);

    free(p);

    p[0] = 'A';
    printf("After free: %c\n", p[0]);

    return 0;
}
```

The pointer is used after the block it refers to has been released.

Checking with Valgrind:

```
trung@trungnm:~/Documents/session2/a$ valgrind --leak-check=full ./dynamic
==31999== Memcheck, a memory error detector
==31999== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==31999== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==31999== Command: ./dynamic
==31999==
a.data = 0x4a7d040
b.data = 0x4a7d040
a.data[0] = A
b.data[0] = A
==31999== Invalid free() / delete / delete[] / realloc()
==31999==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==31999==    by 0x109261: main (dynamic.c:24)
==31999==  Address 0x4a7d040 is 0 bytes inside a block of size 64 free'd
==31999==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==31999==    by 0x109255: main (dynamic.c:23)
==31999==  Block was alloc'd at
==31999==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==31999==    by 0x1091A5: main (dynamic.c:13)
==31999==
==31999==
==31999== HEAP SUMMARY:
==31999==     in use at exit: 0 bytes in 0 blocks
==31999==   total heap usage: 2 allocs, 3 frees, 1,088 bytes allocated
==31999==
==31999== All heap blocks were freed -- no leaks are possible
==31999==
==31999== For lists of detected and suppressed errors, rerun with: -s
==31999== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Bug 4 — Two pointers to the same block

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *a = malloc(64);
    if (a == NULL)
```

```
        return 1;

    a[0] = 'A';
    char *b = a;

    printf("Before free:\n");
    printf("a[0] = %c\n", a[0]);
    printf("b[0] = %c\n", b[0]);

    free(a);
    a = NULL;

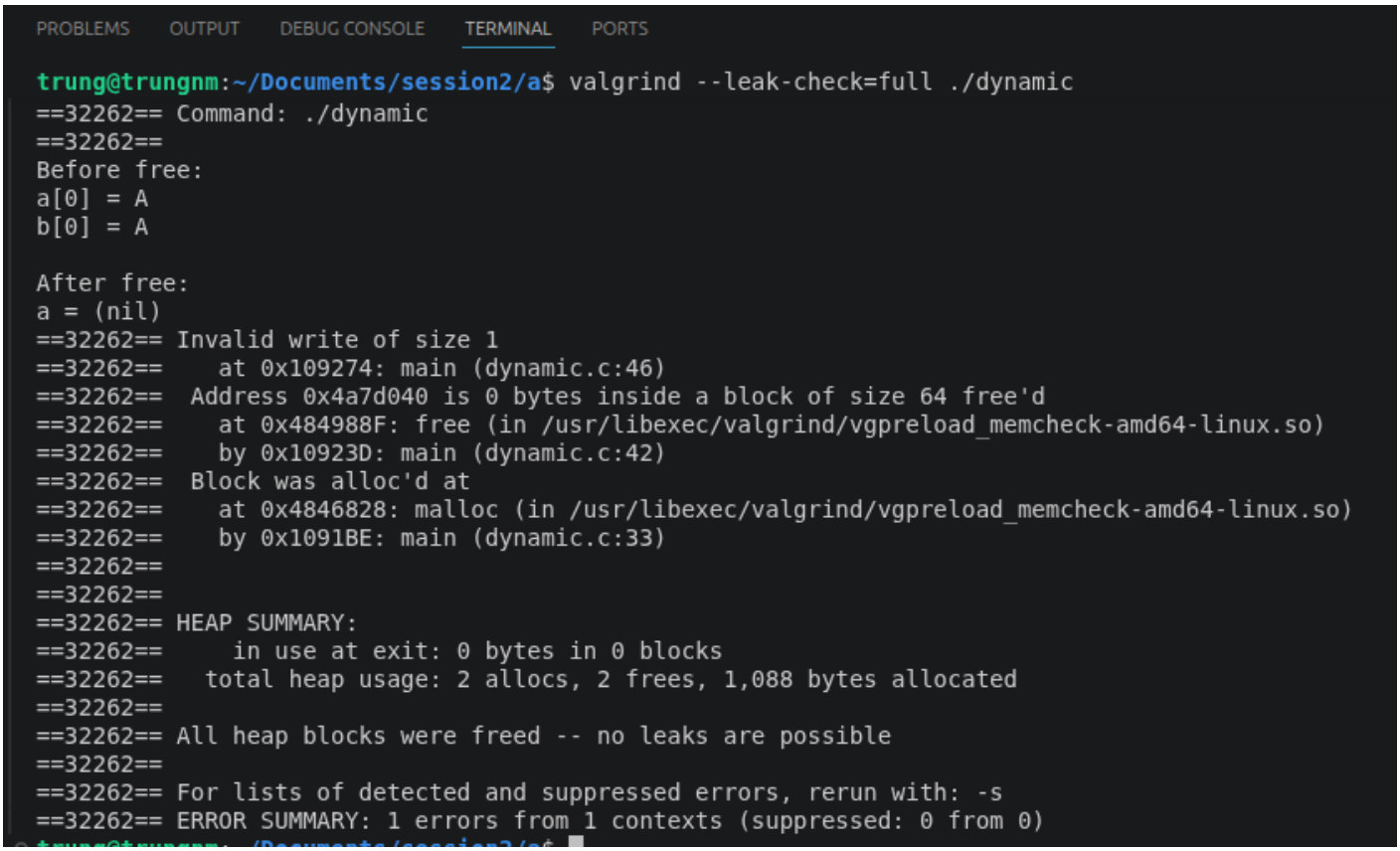
    printf("\nAfter free:\n");
    printf("a = %p\n", (void *)a);

    b[0] = 'X';

    return 0;
}
```

The bug appears when two pointers refer to the same block: setting `a` to `NULL` protects `a`, but `b` still points at freed memory, so writing through `b` is a dangling-pointer write.

Valgrind output:



Bug 5 — Shallow copy leading to a double free

```
#include <stdio.h>
#include <stdlib.h>

struct Buffer {
    int   size;
    char *data;
};

int main(void)
{
    struct Buffer a;

    a.size = 64;
    a.data = malloc(64);
    if (a.data == NULL)
        return 1;

    a.data[0] = 'A';

    struct Buffer b = a;
```

```
    printf("a.data = %p\n", (void *)a.data);
    printf("b.data = %p\n", (void *)b.data);
    printf("a.data[0] = %c\n", a.data[0]);
    printf("b.data[0] = %c\n", b.data[0]);

    free(a.data);
    free(b.data);

    return 0;
}
```

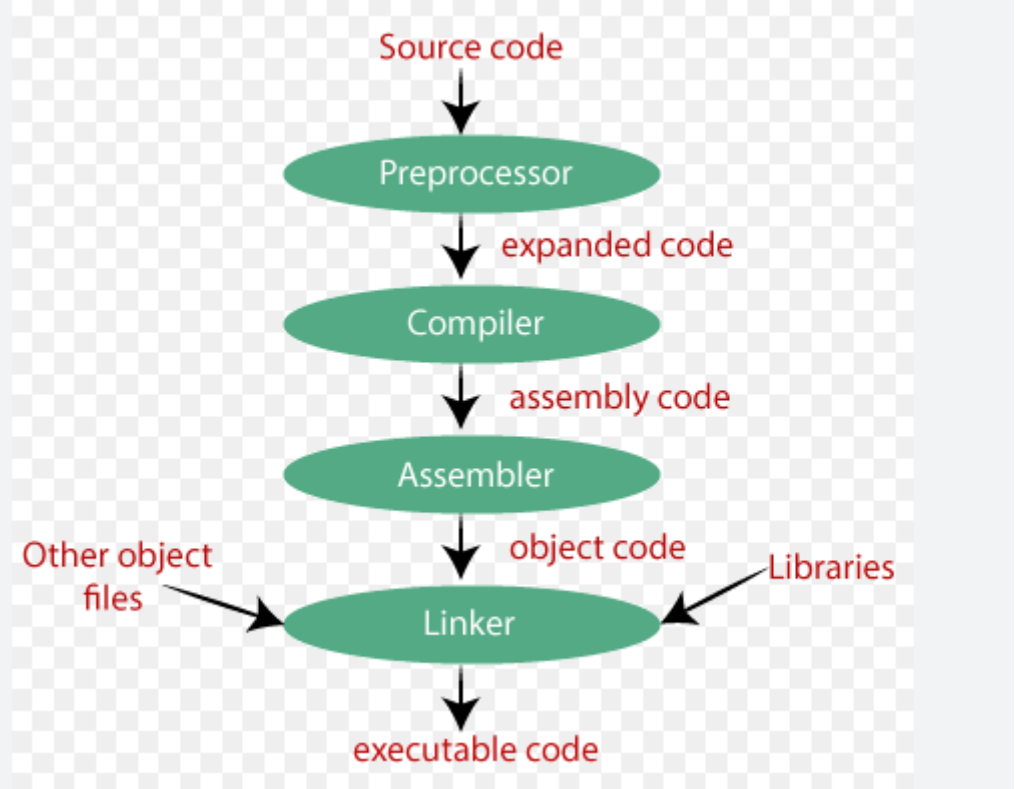
Assigning one struct to another copies the pointer, not the buffer it refers to. Both structs end up owning the same block, and freeing it twice is undefined behavior.

Valgrind output:

```
trung@trungnm:~/Documents/session2/a$ valgrind --leak-check=full ./dynamic
==32374==    by 0x1091E3: main (dynamic.c:63)
==32374==  Block was alloc'd at
==32374==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32374==    by 0x10919E: main (dynamic.c:57)
==32374==
==32374== Invalid read of size 1
==32374==    at 0x1091EF: main (dynamic.c:65)
==32374==  Address 0x4a7d040 is 0 bytes inside a block of size 64 free'd
==32374==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32374==    by 0x1091E3: main (dynamic.c:63)
==32374==  Block was alloc'd at
==32374==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32374==    by 0x10919E: main (dynamic.c:57)
==32374==
After free: A
==32374==
==32374== HEAP SUMMARY:
==32374==     in use at exit: 0 bytes in 0 blocks
==32374==   total heap usage: 2 allocs, 2 frees, 1,088 bytes allocated
==32374==
==32374== All heap blocks were freed -- no leaks are possible
==32374==
==32374== For lists of detected and suppressed errors, rerun with: -s
==32374== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
```

6) Compilation Pipeline Refresher: Preprocessor, Compiler Proper, Assembler, Linker (`gcc -E` / `-S` / `-c`)

Definition. Compilation is the process of turning C source code into an executable file, and it runs through four consecutive stages. Each stage takes the output of the previous one as its input.



Stage 1 — Preprocessing

Handles every line beginning with `#`. It works purely on text and understands nothing of C syntax.

Its main jobs:

- Insert the contents of files named by `#include`
- Expand `#define` macros
- Keep or discard code according to `#if` / `#ifdef`
- Strip comments

Example:

```
#include <stdio.h>

#define MAX 10

int main(void)
{
    printf("%d\n", MAX * 2);
    return 0;
```

```
}
```

Run it with:

```
gcc -E complination.c -o complination.i
```

The result is a `.i` file: the headers have been expanded inline and the macros substituted, so `MAX * 2` becomes `10 * 2`.

Stage 2 — Compiling (compiler proper)

Turns C code into assembly. This is the stage that checks syntax and types and performs optimization.

```
gcc -S complination.c -o complination.s
```

The result is an assembly file:

Note that `MAX * 2` has already been folded into the constant `$20` at compile time.

Stage 3 — Assembling

Converts assembly into machine code and packages it into an object file.

```
gcc -c complination.c -o complination.o
```

The result is a binary object file — the editor cannot display it as text.

Stage 4 — Linking

Combines object files and libraries into a single executable, filling in the real addresses for every reference.

The linker does three things:

- **Symbol resolution.** It finds a definition for every undefined (`U`) name; `printf` is located in the standard library.
- **Section merging.** All `.text` sections from the various `.o` files are merged into one `.text`, and likewise for `.data` and `.bss`.
- **Relocation.** Once the final layout is known, every placeholder is replaced with a real address.

> [!NOTE] Extern is used to declare an object/function that is defined elsewhere. A normal function declaration already has external linkage by default, so extern usually doesn't need to be written for it. But int x; is a tentative definition, not a declaration-only like extern int x;

Errors characteristic of this stage:

```
undefined reference to `foo'      → declared but never defined
multiple definition of `bar'      → defined twice
```

Example run:

```
gcc complination.o -o complination
```

7) Basic Data Structures Refresher: Singly / Doubly Linked Lists, and Simple Array-Based Structures

`sizeof(array)` vs. `sizeof(pointer)`

```
int a[10];
printf("%zu\n", sizeof(a));      /* 40 */
```

```
int *p = a;
printf("%zu\n", sizeof(p));      /* 8  */
```

An array name and a pointer to its first element are not the same thing. `sizeof(a)` yields the size of the whole array — 10 elements × 4 bytes = 40. `sizeof(p)` yields the size of a pointer on the target machine — 8 bytes on a 64-bit system, 4 bytes on a 32-bit MCU — no matter how large the array it points at.

The conversion from array to pointer (**array-to-pointer decay**) happens automatically in almost every expression, but not under `sizeof`, `_Alignof`, or `&`. This is exactly why an array passed to a function loses its length:

```
void f(int arr[10])
{
    printf("%zu\n", sizeof(arr));   /* 8, not 40 — arr is an int * */
}
```

The parameter is adjusted to `int *` (see section 2), so the length must be passed alongside it:

```
void f(int *arr, size_t len);

int a[10];
f(a, sizeof a / sizeof a[0]);       /* compute the count in the caller */
```

The idiom `sizeof a / sizeof a[0]` is only correct where `a` is still an array — in the scope where it was declared. Inside the function it silently computes `8 / 4 = 2`.

Singly Linked List

A singly linked list is a chain of nodes; each node holds its own data plus a pointer to the next node. The last node's `next` is `NULL`. Unlike an array, the nodes are not contiguous in memory, so there is no random access — reaching element *n* requires walking *n* links — but insertion and removal cost only a pointer reassignment.

```
typedef struct Node {
    int          data;
    struct Node *next;
} Node;
```

Note that the tag `struct Node` is required inside the declaration: the `typedef` name does not exist yet at the point where `next` is declared.

Bug — Use After Free When Releasing a List

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int          data;
    struct Node *next;
```

```
} Node;

/* use-after-free */
void free_list_wrong(Node *head)
{
    Node *current = head;

    while (current != NULL) {
        free(current);
        current = current->next;
    }
}

Node *make_node(int data)
{
    Node *n = malloc(sizeof(Node));
    n->data = data;
    n->next = NULL;
    return n;
}

int main(void)
{
    Node *head = make_node(1);
    head->next = make_node(2);
    head->next->next = make_node(3);

    free_list_wrong(head);
    /* free_list_correct(head); */
    head = NULL;

    printf("Done.\n");
    return 0;
}
```

The bug is the order of the two statements inside the loop. `free(current)` releases the node, and only afterwards does `current = current->next` read the `next` field — out of a block that no longer belongs to the program. This is a use-after-free (section 5, bug 3): the read may return the correct address by chance, or garbage once the allocator has reused the block for its own bookkeeping, in which case the loop walks off into an arbitrary address and the program crashes or corrupts the heap.

The fix is to save the next pointer *before* freeing the node:

```
void free_list_correct(Node **head)
{
    Node *current = *head;

    while (current != NULL) {
        Node *next = current->next;   /* read before releasing */
        free(current);
        current = next;
    }

    *head = NULL;                     /* caller's pointer no longer dangles */
}
```

Two further points in the example worth fixing:

- `make_node` does not check the return value of `malloc`. If allocation fails, `n->data = data` dereferences `NULL`.
- Passing `head` by value into `free_list_wrong` means the function cannot clear the caller's pointer; the assignment `head = NULL` in `main` has to be remembered manually. Taking `Node **`, as in `free_list_correct`, makes that impossible to forget.

Verify with Valgrind — the invalid read is reported at the `current = current->next` line:

```
gcc -g -o list list.c
valgrind --leak-check=full ./list
```

Doubly Linked List

Each node also stores a pointer back to its predecessor, so the list can be traversed in both directions and a node can be unlinked without first walking from the head to find the one before it.

```
typedef struct DNode {
    int           data;
    struct DNode *prev;
    struct DNode *next;
} DNode;
```

Removing a node means repairing both neighbours, guarding for the ends of the list:

```
void unlink_node(DNode **head, DNode *node)
{
    if (node->prev != NULL)
        node->prev->next = node->next;
    else
        *head = node->next;           /* node was the first element */

    if (node->next != NULL)
        node->next->prev = node->prev;

    free(node);
}
```

The trade-off against a singly linked list is one extra pointer per node — 8 bytes on a 64-bit host — and one more field to keep consistent on every insertion and removal.

Simple Array-Based Structures

For a fixed, known upper bound, an array beats a linked list on almost every axis that matters in firmware: no allocator, no per-node pointer overhead, and contiguous memory that the cache and the prefetcher handle well.

A ring buffer is the typical example — a fixed array plus two indices:

```
#include <stdint.h>
#include <stdbool.h>

#define RB_SIZE 16                    /* power of two: & wraps faster than % */

typedef struct {
    uint8_t  buf[RB_SIZE];
    uint16_t head;                    /* write index */
    uint16_t tail;                    /* read index  */
} RingBuffer;

bool rb_push(RingBuffer *rb, uint8_t byte)
{
    uint16_t next = (rb->head + 1) & (RB_SIZE - 1);

    if (next == rb->tail)             /* full */
```

```
        return false;

    rb->buf[rb->head] = byte;
    rb->head = next;
    return true;
}

bool rb_pop(RingBuffer *rb, uint8_t *out)
{
    if (rb->head == rb->tail)         /* empty */
        return false;

    *out = rb->buf[rb->tail];
    rb->tail = (rb->tail + 1) & (RB_SIZE - 1);
    return true;
}
```

One slot is deliberately left unused so that "full" and "empty" remain distinguishable — otherwise `head == tail` would mean both.

8) Function pointers and callback-style APIs

Mistake 1: Casting a function pointer, then calling it

The problem

Calling a function through a pointer of the wrong type is undefined behavior.

The cast compiles silently. But at runtime, the call passes arguments one way while the function body reads them another way.

Example

```
#include <stdio.h>

void takes_double(double d) { printf("d = %f\n", d); }
void takes_two(int a, int b) { printf("a = %d, b = %d\n", a, b); }

int main(void)
{
    ((void (*)(int))takes_double)(1);   /* wrong parameter type  */
    ((void (*)(int))takes_two)(10);     /* wrong parameter count */
    return 0;
}
```

```
d = 0.000000
a = 10, b = 32767
```

The value `1` goes into an integer register, but `takes_double` reads from a float register. And `takes_two` reads a second argument that was never passed.

No warning is produced. An explicit cast tells the compiler you know what you are doing.

Mistake 2: Checking NULL in the wrong place

The problem

A function name is never NULL. Only a function pointer variable can be NULL.

Two directions

Direction 1 — checking something that is never NULL:

```
void handler(void) { }
```

```
if (handler)          /* always true  */
    handler();

if (handler == NULL)  /* always false — dead branch */
    fallback();
```

The `fallback()` branch never runs. The code looks like it handles an error but does not.

Direction 2 — forgetting to check something that can be NULL:

```
static void (*g_cb)(void);   /* starts as NULL */

void fire_event(void)
{
    g_cb();                  /* crashes if nobody registered */
}
```

Example

```
#include <stdio.h>

void handler(void) { printf("handler called\n"); }

static void (*g_cb)(void);   /* NULL */

int main(void)
{
    if (handler)
        printf("if (handler) -> always true\n");

    if (handler == NULL)
        printf("this line never prints\n");

    printf("calling g_cb...\n");
    g_cb();                  /* segfault */

    return 0;
}
```

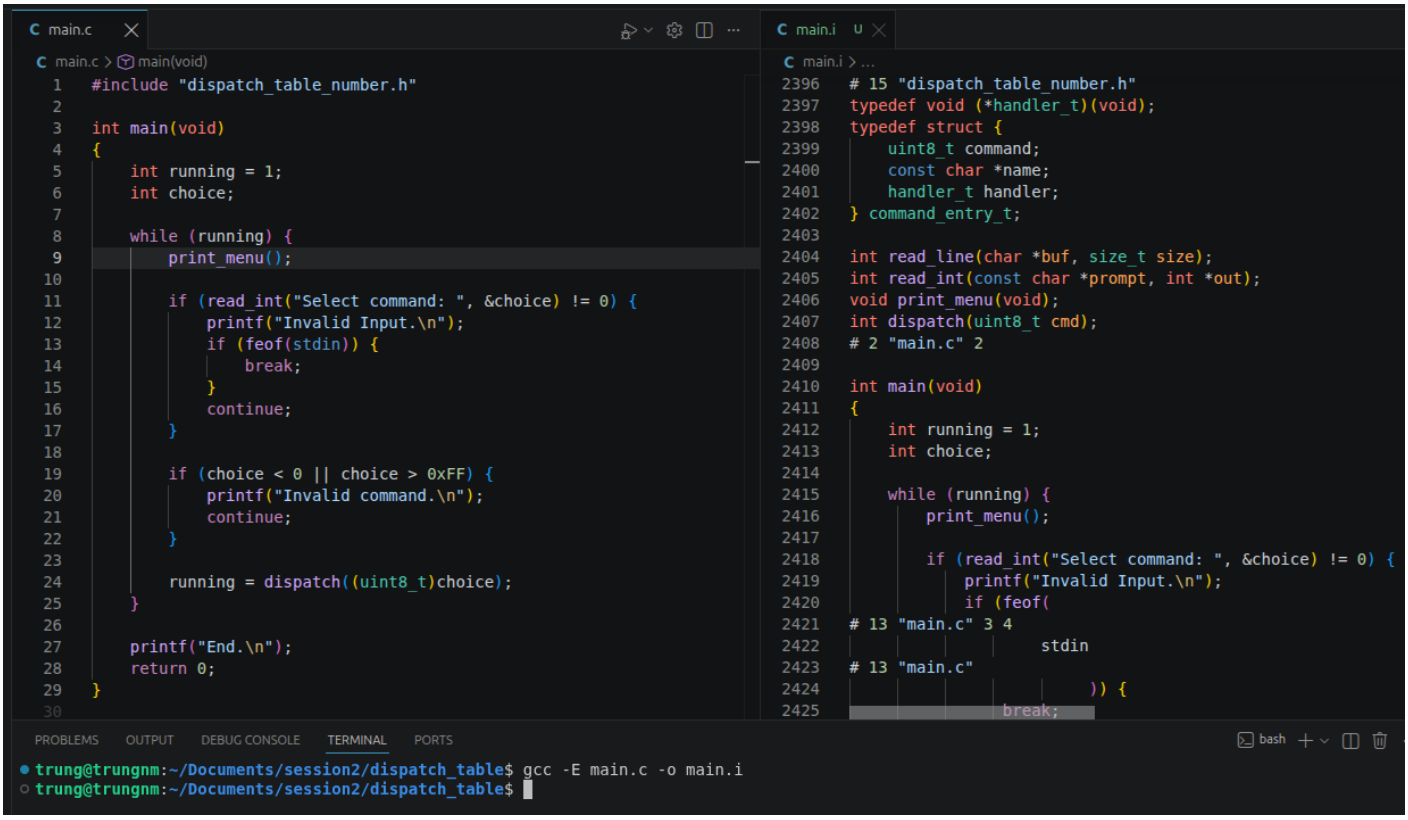
```
if (handler) -> always true
calling g_cb...
Segmentation fault
```

It usually appears during refactoring. Someone changes a function pointer variable into a fixed function — and the NULL check silently becomes meaningless. Or the reverse: a fixed function becomes a pointer, and nobody adds the check.

Lab 1 — LinkList

Lab Linklist

Lab 2 — Compilation Process



main.c

```c
#include "dispatch_table_number.h"

int main(void)
{
    int running = 1;
    int choice;

    while (running) {
        print_menu();

        if (read_int("Select command: ", &choice) != 0) {
            printf("Invalid Input.\n");
            if (feof(stdin)) {
                break;
            }
            continue;
        }

        if (choice < 0 || choice > 0xFF) {
            printf("Invalid command.\n");
            continue;
        }

        running = dispatch((uint8_t)choice);
    }

    printf("End.\n");
    return 0;
}
```

main.s

```
    .file   "main.c"
    .text
    .section    .rodata
.LC0:
    .string "Select command: "
.LC1:
    .string "Invalid Input."
.LC2:
    .string "Invalid command."
.LC3:
    .string "End."
    .text
    .globl  main
    .type   main, @function
main:
.LFB6:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    subq    $16, %rsp
    movq    %fs:40, %rax
    movq    %rax, -8(%rbp)
    xorl    %eax, %eax
    movl    $1, -12(%rbp)
    jmp .L2
.L9:
```

```
trung@trungnm:~/Documents/session2/dispatch_table$ gcc -E main.c -o main.i
trung@trungnm:~/Documents/session2/dispatch_table$ gcc -S main.i -o main.s
trung@trungnm:~/Documents/session2/dispatch_table$
```

dispatch_table_number.c

```c
#include "dispatch_table_number.h"

static int    g_numbers[MAX_NUMBERS];
static size_t g_count = 0;

int read_int(const char *prompt, int *out)
{
    char  buf[INPUT_BUF_SZ];
    char *end;
    long  val;

    printf("%s", prompt);
    fflush(stdout);

    if (read_line(buf, sizeof(buf)) != 0)
    {
        return -1;
    }

    if (buf[0] == '\0')
    {
        return -1;
    }

    errno = 0;
    val = strtol(buf, &end, 10);

    if (*end != '\0')
    {
        return -1;
    }
```

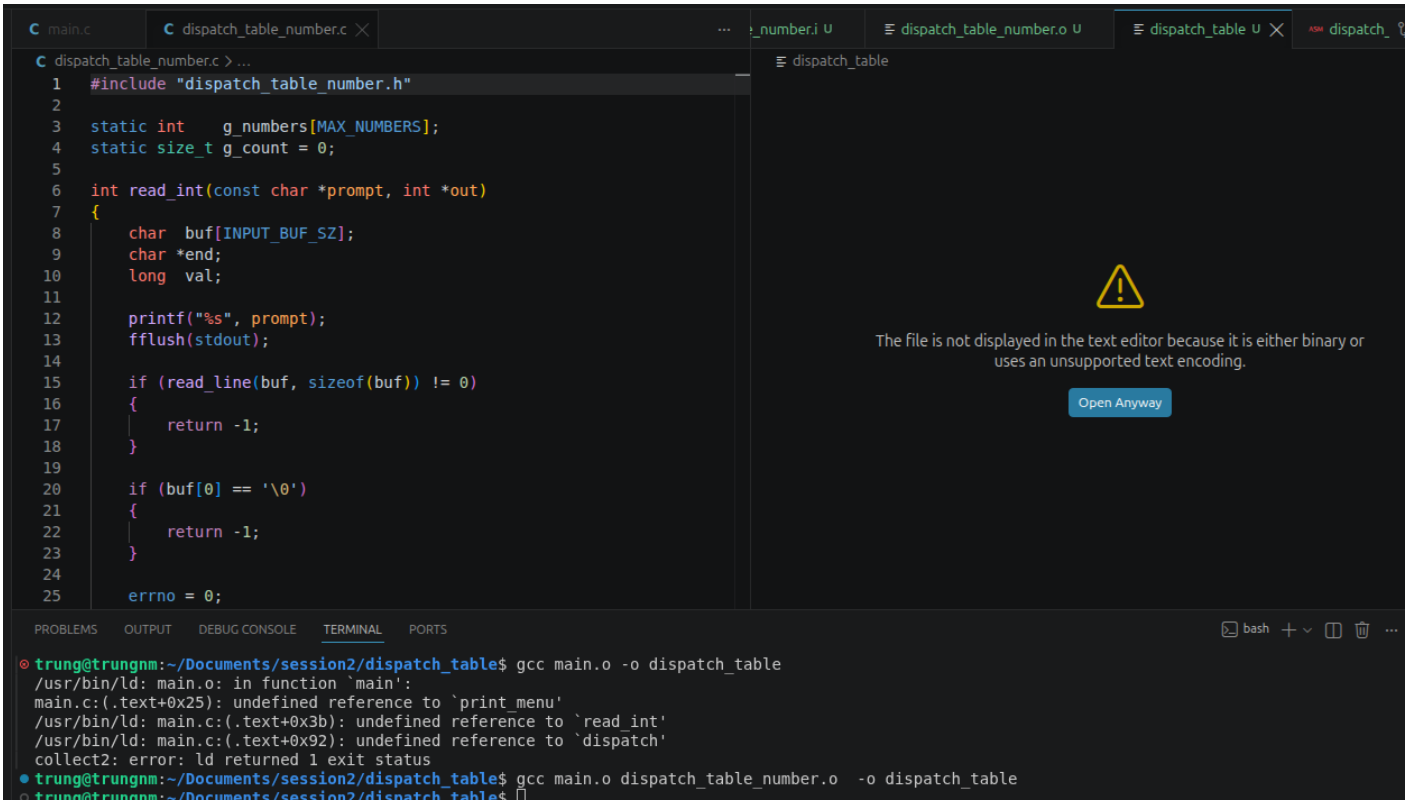
dispatch_table_number.o



The file is not displayed in the text editor because it is either binary or uses an unsupported text encoding.

Open Anyway

```
trung@trungnm:~/Documents/session2/dispatch_table$ as dispatch_table_number.s -o dispatch_table_number.o
trung@trungnm:~/Documents/session2/dispatch_table$
```



Errors characteristic of this stage:

```
/usr/bin/ld: main.o: in function `main':
main.c:(.text+0x25): undefined reference to `print_menu'
/usr/bin/ld: main.c:(.text+0x3b): undefined reference to `read_int'
/usr/bin/ld: main.c:(.text+0x92): undefined reference to `dispatch'
collect2: error: ld returned 1 exit status
```

The error occurs because only main.o was linked. The required functions are defined in dispatch_table_number.o, but this object file was not provided to the linker. Therefore, during the symbol resolution stage, the linker cannot find the definitions of print_menu, read_int, and dispatch, resulting in undefined reference errors.

Lab 3 — Dispatch Table

Lab Dispatch Table

Revision #7
Created 2026-08-19 07:13:12 UTC by trungnguyen
Updated 2026-09-03 02:13:49 UTC by trungnguyen